

Signature of the Student: A. Pavan Charan						
Q. No	Understanding of Concepts (25)	Experimental Design (30)	Use of Tools (20)	Analysis of Results (15)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						

	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Annexure A

EXPERIMENTS

Q1. Write a Lex program to recognize uppercase letters (A–Z) from the given input. The program should identify and count all capital letters present in the string. Display the total number of uppercase letters detected.

Answer:

Uppercase Letter Recognition

Aim

To develop a Lex-based web application that recognizes uppercase letters **A–Z** from a given input string and counts the total number of uppercase letters.

Algorithm

1. Start the program.
2. Read the input string.
3. Initialize the uppercase counter to 0.
4. Scan each character of the input string.
5. Check whether the character lies between A and Z.
6. If true, increment the uppercase counter.
7. Continue until all characters are processed.
8. Display the total number of uppercase letters.
9. Stop.

Pseudocode

START

```
READ input string
SET uppercase_count = 0
FOR each character ch in input string
  IF ch >= 'A' AND ch <= 'Z'
    uppercase_count = uppercase_count + 1
  END IF
END FOR
DISPLAY uppercase_count
STOP
```

Working

The application accepts a string and examines every character individually. If a character belongs to the range A–Z, it is identified as an uppercase letter and the counter is incremented.

Input

Hello WORLD Compiler Design

Output



The screenshot shows a web application titled "UPPERCASE LETTER ANALYZER". It features a text input field containing "Hello WORLD 123 @ Compiler DESIGN". Below the input field, there are two buttons: "Analyze" (green) and "Clear" (red). To the right of the input field, it says "Characters Typed : 33". Below the buttons, there are five boxes displaying counts: "Characters" (33), "Uppercase" (13), "Lowercase" (11), "Digits" (3), and "Symbols" (1). Below these boxes, it shows "Uppercase Percentage: 39.4%" with a green progress bar. At the bottom, there is a section titled "Highlighted Uppercase Letters" showing the text "H W O R L D C O D E S I G N" where the uppercase letters are highlighted in green.

Result

Characters : 27

Uppercase : 11

Lowercase : 14

Digits : 0

Symbols : 2

Q2. Write a Lex program to validate an email address entered by the user. The program should check for the presence of a valid username, '@' symbol, and domain name. Display whether the email address is valid or invalid.

Answer:

Email Validation

Aim

To develop a Lex-based application to validate an email address by checking the username, @ symbol, and domain name, and display whether the email address is valid or invalid.

Algorithm

1. Start the program.
2. Read the email address.
3. Separate the username and domain using the @ symbol.
4. Check whether a valid username exists.
5. Check whether exactly the required email structure is present.
6. Check whether a valid domain exists.
7. If all conditions are satisfied, display **Valid Email**.
8. Otherwise, display **Invalid Email**.
9. Stop.

Pseudocode

START

READ email

CHECK username

CHECK '@' symbol

CHECK domain

IF username is valid

AND '@' symbol exists

```
AND domain is valid
THEN
  DISPLAY "VALID EMAIL"
ELSE
  DISPLAY "INVALID EMAIL"
END IF
STOP
```

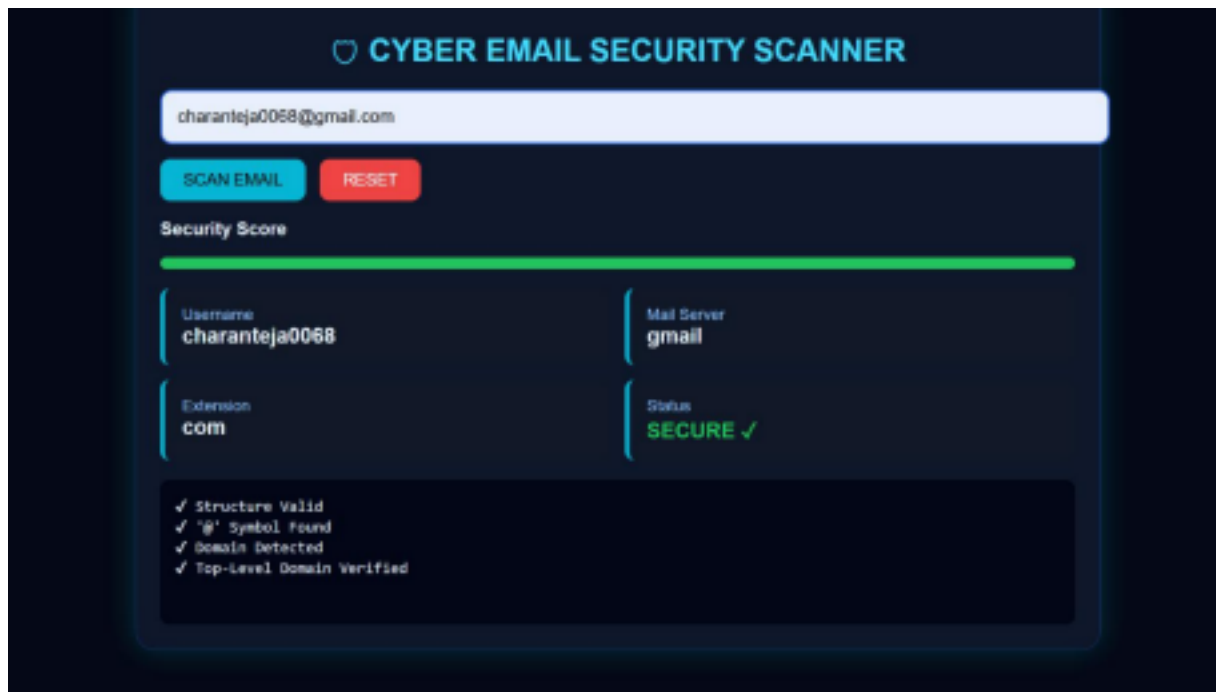
Process / Working

The entered email is analyzed according to the expected email structure: The application extracts the username, domain, and top-level domain and then validates the complete structure.

Input

student@gmail.com

Output



Result

Username : student

Domain : gmail

TLD : com

Status : VALID

Q3. Write a Lex program to recognize and validate a date of birth in the format DD/MM/YYYY. The program should check whether the entered date follows the specified format. Display a message indicating whether the date of birth is valid or invalid.**Q4.** You are building a type checker in a compiler.

Answer:

Date of Birth Validation

Aim

To develop a Lex-based application to recognize and validate a date of birth in the format **DD/MM/YYYY** and display whether the entered date is valid or invalid.

Algorithm

1. Start the program.
2. Read the date of birth.
3. Check whether the input follows DD/MM/YYYY.
4. Separate day, month and year.
5. Verify whether the date is a valid calendar date.
6. Otherwise, display an invalid-date message.
7. Stop.

Pseudocode

START

READ DOB

CHECK format DD/MM/YYYY

IF format is invalid

 DISPLAY "INVALID DATE"

STOP

END IF

EXTRACT day, month, year

CREATE date using day, month, year

```
IF created date matches  
original day, month and year  
THEN  
  DISPLAY "VALID DATE"  
ELSE  
  DISPLAY "INVALID DATE"  
END IF  
STOP
```

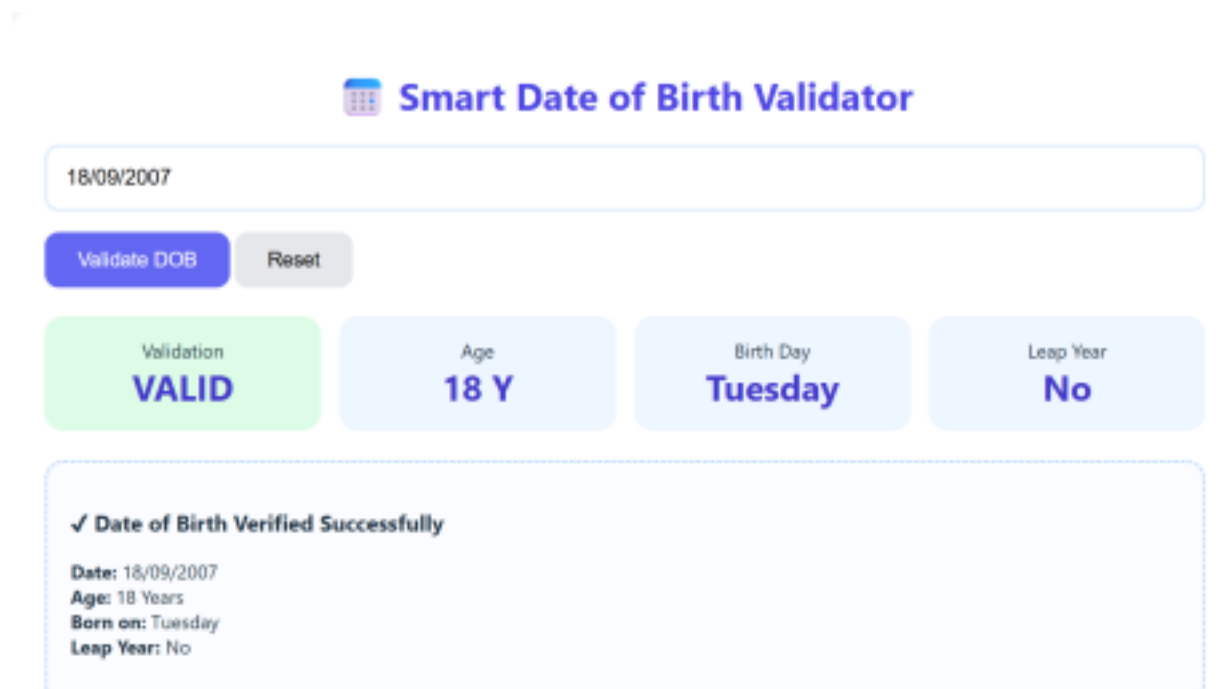
Working

The application first verifies the format and then checks whether the entered day actually belongs to the specified month and year.

Input

18/09/2007

Output



The screenshot shows a web application titled "Smart Date of Birth Validator". It features a date input field containing "18/09/2007". Below the input are two buttons: "Validate DOB" (highlighted in blue) and "Reset". The results are displayed in four colored boxes: a green box for "Validation" showing "VALID", a light blue box for "Age" showing "18 Y", a light blue box for "Birth Day" showing "Tuesday", and a light blue box for "Leap Year" showing "No". At the bottom, a light blue box with a dashed border contains a success message: "✓ Date of Birth Verified Successfully", followed by a list of details: "Date: 18/09/2007", "Age: 18 Years", "Born on: Tuesday", and "Leap Year: No".

Result

Validation : VALID

Age : 18 Years

Birth Day : Tuesday

Leap Year : No

Q4. Write a Lex program to perform token recognition in a source program. The program should identify keywords, identifiers, operators, separators, and numeric constants. Display the type of each token recognized from the input.

Answer:

Token Recognition

Aim

To develop a Lex-based token recognition application that identifies **keywords, identifiers, operators, separators and numeric constants** from a source program and displays the type of each token.

Algorithm

1. Start the program.
2. Read the source program.
3. Scan the source code from left to right.
4. Identify keywords.
5. Identify identifiers.
6. Identify numeric constants.
7. Identify operators.
8. Identify separators.
9. Display each token with its corresponding type.
10. Stop.

Pseudocode

START

READ source program

WHILE input is not completely processed

IF token is a keyword

DISPLAY token, "KEYWORD"

ELSE IF token is an identifier

DISPLAY token, "IDENTIFIER"

ELSE IF token is a number
DISPLAY token, "NUMERIC CONSTANT"
ELSE IF token is an operator
DISPLAY token, "OPERATOR"
ELSE IF token is a separator
DISPLAY token, "SEPARATOR"
END IF
END WHILE
STOP

Working

The source program is scanned token by token. Each token is compared with the defined lexical patterns and classified into its corresponding category.

Input

```
int a = 10;  
float b = 25.5;  
if(a > b)  
{  
    printf("Hello");  
}
```

Output

The screenshot shows a software interface titled "Compiler Token Recognition Studio". It is divided into two main sections: "Source Code" on the left and "Token Table" on the right.

Source Code:

```
int a = 10;
float b = 25.5;
if(a>b){
printf("Hello");
}
```

Token Table:

Token	Type
int	Keyword
a	Identifier
=	Operator
10	Number
;	Separator
float	Keyword
b	Identifier
=	Operator
25.5	Number
;	Separator
if	Keyword
{	Separator
a	Identifier
>	Operator
b	Identifier
}	Separator
;	Separator
printf	Keyword
(	Separator
hello	Identifier

Result

The program executed successfully without any error using HTML

Q5. Write a Lex program to identify vowels and consonants in a given string. The program should count the number of vowels and consonants separately. Display the total counts after processing the input string.

Answer:

Vowel and Consonant Recognition

Aim

To develop a Lex-based application that identifies vowels and consonants in a given string and counts them separately.

Algorithm

1. Start the program.
2. Read the input string.
3. Initialize vowel_count and consonant_count to 0.
4. Scan every character.
5. Check whether the character is an alphabet.
6. If it is A, E, I, O, U or their lowercase forms, increment the vowel count.
7. Otherwise, classify it as a consonant and increment the consonant count.
8. Ignore spaces, digits and special characters.
9. Display the vowel and consonant counts.

10. Stop.

Pseudocode

START

READ input string

SET vowel_count = 0

SET consonant_count = 0

FOR each character ch

IF ch is an alphabet

IF ch is A, E, I, O, U

OR ch is a, e, i, o, u

vowel_count = vowel_count + 1

ELSE

consonant_count = consonant_count + 1

END IF

END IF

END FOR

DISPLAY vowel_count

DISPLAY consonant_count

STOP

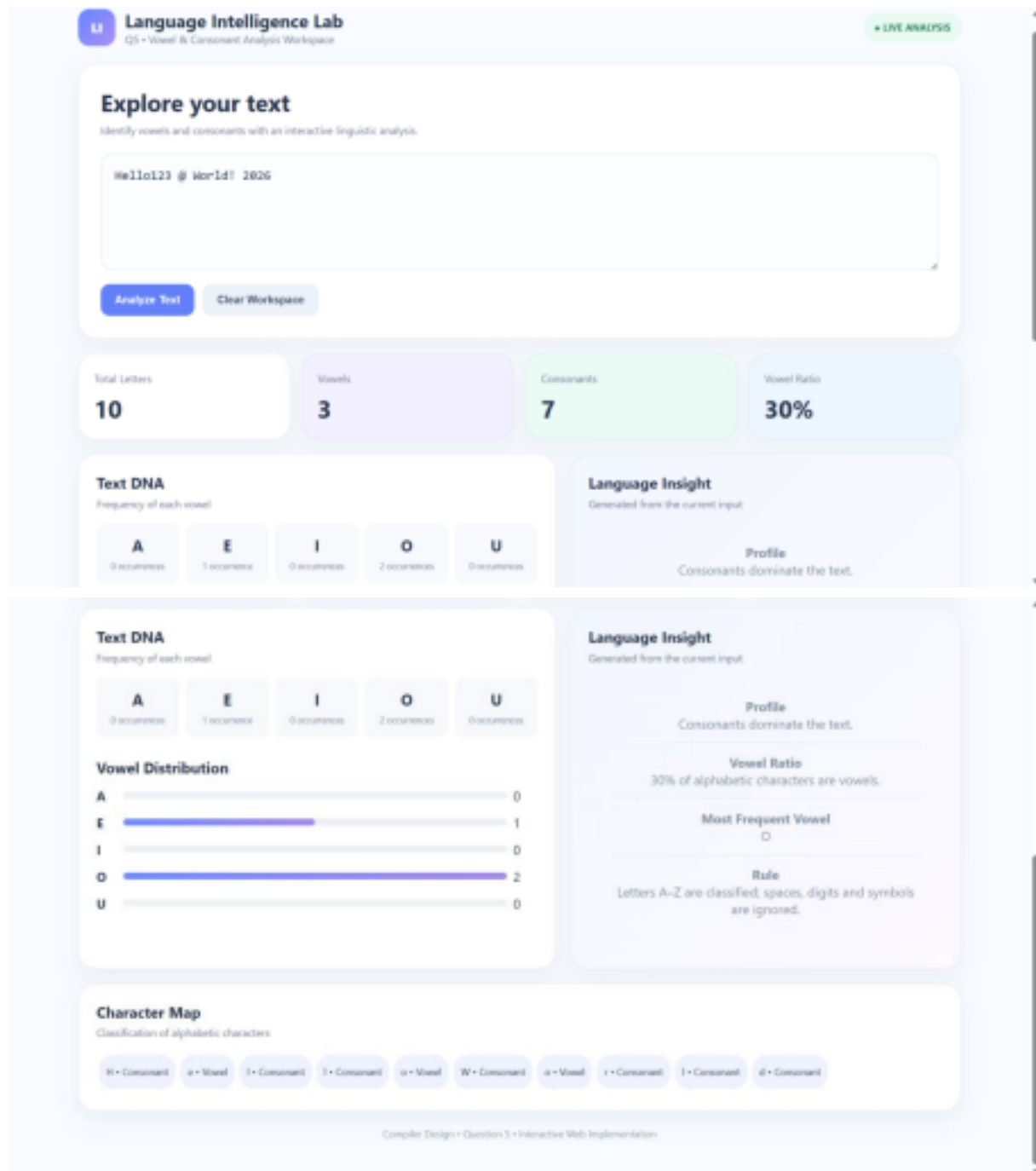
Working

The application scans the string character by character. Alphabetic characters are classified as vowels or consonants. Non-alphabetic characters such as spaces, numbers and symbols are ignored.

Input

The Quick Brown Fox Jumps Over The Lazy Dog

Output



Result

Total Letters : 35

Vowels : 11

Consonants : 24

Vowel Ratio : 31%

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation